

考场规则

笔，橡皮可以带进考场，手机无论开关机都不能带进考场，草稿纸考点会提供，也不能带进考场，也不能自备鼠标键盘。

信息学竞赛

全国青少年信息学奥林匹克联赛(NOIP)是从1995年开始的，到2018年一共举办了24届。

全国青少年计算机程序设计竞赛(NOI)是从1984年开始的，邓小平提出计算机要从娃娃抓起的那一年。

NOIP从2022年开始禁止使用Pascal

哈夫曼树

带权路径最短的二叉树，又称最优二叉树，其树中的结点只有叶子结点和度数为2的结点。哈夫曼树的构建采用了贪心的原理。

冯. 诺依曼理论

计算机硬件设备由**存储器，运算器，控制器，输入设备，输出设备**5个部分组成。CPU既是运算器又是控制器。

存储器

存储器分为主存储器(内存)和辅助存储器(外存)，主存储器是临时存储器，辅助存储器是永久存储器。内存存取速度快，容量小，内存分为ROM，RAM，cache, 其中cache(高速缓存)读取速度最快，最接近CPU。寄存器在CPU中，所以寄存器的速度最快。

外存 $\longrightarrow$ 内存(RAM) $\longrightarrow$ cache $\longrightarrow$ CPU

计算机奖项：

属于计算机科学领域的奖项有图灵奖，王选奖，图灵奖被称为“计算机界的诺贝尔奖”，是美国计算机协会 (ACM) 为了纪念英国人“艾伦. 麦席森. 图灵”所创立的

操作系统

常见操作系统有Windows, Unix, Linux, Mac OS

软件分为系统软件和应用软件

计算机语言：

计算机语言分为机器语言，汇编语言，高级语言，机器语言就是计算机能直接识别的语言，又称二进制语言，是最早期的语言。后面出现了汇编语言，不能被机器直接识别，必须要使用某种特殊软件转换成机器语言，属于低级语言。现在人们常用的是高级语言，高级语言就是我们人们常用的语言，目前一般是英文。比如现在常用的C++，C，pascal，VB这些都是高级语言。高级语言需要用编译器转换之后才能被电脑识别。

程序设计语言可以分为两种，面向对象程序设计，面向过程程序设计。

比如C++，C#，JAVA，python，VB是面向对象语言，面向过程的语言比较少，常见的就是C语言，Pascal。

还可以分为编译性语言和解释性语言。

C, C++, Pascal都是编译性语言，修改源程序后必须要重新编译。
其他的ASP, PHP, PYTHON都是解释性语言

进制转换

十进制：逢十进一(包含数字0--9)

$$(365)_{10} = 3*10^2 + 6*10^1 + 5*10^0$$

二进制：逢二进一(包含数字0-1)

$$(1011)_2 = 1*2^3 + 0*2^2 + 1*2^1 + 1*2^0$$

二进制转十进制：按位展开

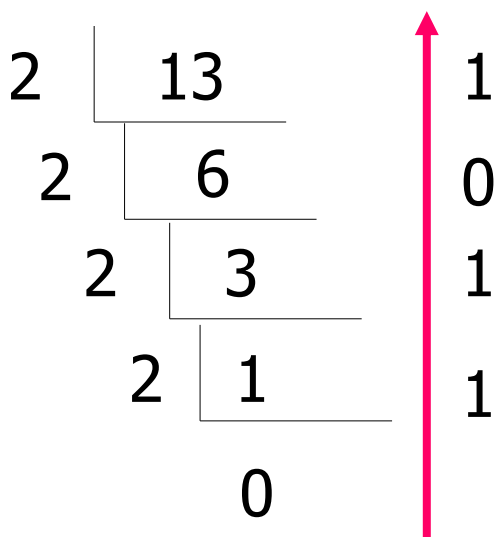
$$(1011)_2 = 1*2^3 + 0*2^2 + 1*2^1 + 1*2^0 = 11$$

进制转换

十进制转二进制：

反过来，除二倒取余。

例：十进制数13转化成二进制数



结果为：1101

进制转换

如果是m进制数转n进制数

直接转换有公式，但是太麻烦，统一先把m进制数转换成十进制数，再把十进制数转换成n进制数。（和二转十，十转二相同）

二进制转4, 8, 16进制可以快速转换。

如2转8， $2^3=8$ ，所以每3位二进制可以看做一位8进制。
反过来也是一样

1	1	1	1	0	1	0	1	1	0	1
3			6			5			5	

从后往前，位数不足，
最前面补0

进制转换

整数的进制转换会了，那么如果遇到小数的进制转换怎么办呢？其实也很简单。

小数二进制转十进制：

整数部分是 $\times 2$ 的 $\dots 3, 2, 1, 0$ 次方，那么小数部分照旧，按照位数继续乘以2的 $-1, -2, -3 \dots$ 次方就可以了。

比如 $0.0101 = 1 \times 2^{-2} + 1 \times 2^{-4}$

小数十进制转二进制也差不多，以前是 $/2$ ，现在变成 $\times 2$ ，比如0.25转换成二进制。 $0.25 \times 2 = 0.5$ 取整数位0， $0.5 \times 2 = 1$ ，取整数位1，小数位没有了就不管了，最终的答案就是.01。比如0.4转换成二进制就是无限循环形式。

进制转换

初赛的时候小数转换选择题其实很简单的，因为要显示出来那么肯定不能是无限循环小数。

所以对于2转10，一般都是0.1, 0.11之类的，很简单，二进制的0.2相当于就是十进制的1.0，那么0.1就是0.2的一般，对应十进制1.0的一半就是0.5.

对于10转2，如果要是有限小数，一般都是0.125, 0.25, 0.5之类的，和刚才一样0.25是1的1/4，所以答案就是0.01。

位，字节，字长

位(bit)：计算机中最小的数据单位，存放一位二进制数，他也是存储信息的最小单位，简写为“b”。

字节(Byte)：字节是计算机中存储容量的基本单位，一个字节由8位二进制数组成，一个字符占一个字节，一个汉字占两个，int占4个字节。

字长：字长主要由CPU类型决定，即计算机中一次作为整体被处理的二进制数，目前主流的计算机字长有32位，64位。

1B=8b

1KB=1024B

1MB=1024KB

原码、反码、补码

机器数：一个数在计算机中的二进制表示形式，叫做这个数的机器数，机器数是带符号的，在计算机用一个数的最高位存放符号，**正数为0，负数为1**。

比如计算机字长为8位，十进制3表示出来的机器数就是00000011，十进制数-3表示出来就是10000011

原码：就是机器数

$$[+1]_{\text{原}} = 0000\ 0001$$

$$[-1]_{\text{原}} = 1000\ 0001$$

反码：正数的反码是其本身，负数的反码是在其原码的基础上，符号位不变，其余各位取反。

$$[+1] = [00000001]_{\text{原}} = [00000001]_{\text{反}}$$

$$[-1] = [10000001]_{\text{原}} = [11111110]_{\text{反}}$$

原码、反码、补码

补码：正数的补码是其本身，负数的补码是其反码+1。

$$\begin{aligned} [+1] &= [00000001]_{\text{原}} = [00000001]_{\text{反}} = [00000001]_{\text{补}} \\ [-1] &= [10000001]_{\text{原}} = [11111110]_{\text{反}} = [11111111]_{\text{补}} \end{aligned}$$

总体来说，正数的所有码都是它本身，负数的补码，反码按照规则来计算。至于为什么有原码，反码，补码，仅作了解，反正也不考，大概就是计算机只能做加法，并且可以直接算上符号位做加法，所以原码有问题，就有了反码，反码0有两种形式，就有了补码。

真题

在8位二进制补码中，10101011表示的数是十进制下的（）：

A:43

B:-85

C:-43

D:-84

真题

下列四个不同进制的数中，与其它三项数值上不相等的是

A. $(269)_{16}$

B. $(617)_{10}$

C. $(1151)_8$

D. $(1001101011)_2$

题解

一个二进制数的最高位是符号位，无论是反码，补码，原码，符号都是一样的，不会发生改变。

10101011，最高位为1，表示是负数。

正数的原码，反码，补码均相同，负数

原码除符号位外都取反=反码

反码+1=补码

10101011是补码，转换成原码就反过来计算

$10101011(\text{补}) = 10101010(\text{反}) = 11010101(\text{原})$

$1010101(\text{原}) = 85$

所以对应的数是-85

存储空间

分辨率为1600*900, 16位色的位图, 存储图像信息所需要的空间为()。

1:2812.5KB 2:4218.75KB 3:4320KB 4:2880KB

存储空间

我们电脑显示器实际上是由很多个小点组成的，只是点很小，所以我们肉眼看不到。1600*900是指显示器横向有1600个点，纵向有900个点。

16位表示每个点可以用16位的代码来表示颜色，颜色最多有 2^{16} 种，实际上计算值中的颜色就是用二进制数来表示的。

所以所需空间为

$$1600*900*16=x\text{位}=x/8\text{ B}=x/8/1024\text{ KB}=2812.5\text{KB}$$

一定要注意是/1024(2^{10})而不是1000.

时间计算

2017年10月1日是星期日，1949年10月1日是()

这个自己去算，注意闰年366天，平年365天，如果不是整百，不但要能整除4，还要整除400，非整百年只要能整除4就可以了

运算表达式

运算表达式有三种，前缀表达式，中缀表达式，后缀表达式，我们常用的是中缀表达式，即符号在数字之间，运算符有优先级，人肉眼看着很方便，计算也比较方便。

但是脑补一下就会发现计算机计算这种表达式很麻烦，不信你自己写个代码试试，计算机比较好识别的是后缀表达式，即运算符号在数字后面，遇到符号就把最近的两个数字计算，这样运算符之间没有优先级。

那么这里就涉及到两个知识

- (1) 中缀表达式转后缀表达式(反过来)
- (2) 后缀表达式计算

中缀变后缀

运算表达式有三种，前缀表达式，中缀表达式，后缀表达式，我们常用的是中缀表达式，即符号在数字之间，运算符有优先级，人肉眼看着很方便，计算也比较方便。

但是脑补一下就会发现计算机计算这种表达式很麻烦，不信你自己写个代码试试，计算机比较好识别的是后缀表达式，即运算符号在数字后面，遇到符号就把最近的两个数字计算，这样运算符之间没有优先级。

那么这里就涉及到两个知识

- (1) 中缀表达式转后缀表达式(反过来)
- (2) 后缀表达式计算

中缀变后缀

转换规则：

- (1) 遇到数字的时候直接存入一个新的数组
- (2) 遇到“(”的时候，无论此时栈顶元素为什么，都入栈
- (3) 遇到)”的时候，无论此时栈顶元素为什么，都出栈。依次把栈顶的元素存入新的数组，直到“(”出栈为止。“("不放入数组。
- (4) 遇到“*”和“/”的时候，如果栈为空或者栈顶元素是左括号。入栈。如果优先级大于栈顶元素，入栈(乘除的优先级>加减)，如果优先级小于等于栈顶元素，出栈，直到栈顶元素优先级小于当前字符优先级的时候。当前字符入栈。
- (5) 遇到“+”和“-”的时候。如果栈为空或者栈顶元素为左括号，入栈，否则出栈（因为加减的优先级最低，无论栈顶是什么运算符，优先级都大于等于他，都要出栈），当前字符入栈。
- (6) 字符串遍历完之后，一次出栈，并存储。

栈:












当栈为空的时候，所以运算符入栈

9									
---	--	--	--	--	--	--	--	--	--

+

栈:



100

100

100

5

(



数组：

中缀变后缀

栈:

比如: $9+(3-1)*3+10/2$

(5) “-”，此时栈顶元素为“（”，入栈

当栈顶元素是“（”的时候，当前字符只要不是“）”，都入栈

数组:



中缀变后缀

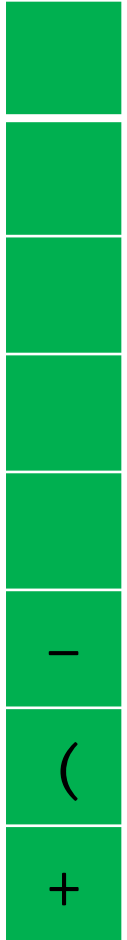
栈:

比如: $9 + (3 - 1) * 3 + 10 / 2$

(6) 数字1, 依次放入新的数组

数字直接放入新的数组

数组:



栈:

+
+
+
+
+
+
+

遇到 “)””，出栈到 “(” 为止。

9	3	1	-						
---	---	---	---	--	--	--	--	--	--

+

栈:

*	
+	

栈顶元素低于当前元素优先级，入栈。

9	3	1	-						
---	---	---	---	--	--	--	--	--	--

+

栈:



100

100

100

5



10

+

中缀变后缀

栈:

比如: $9 + (3 - 1) * 3 + 10 / 2$

(10) “+”，“+”的优先级不高于“*”，栈顶元素“*”出栈，此时栈顶是“+”，优先级也不高于“+”，出栈，此时栈为空，“+”入栈

当前元素优先级不高于栈顶元素，栈顶元素入栈，直到栈顶元素优先级小于当前元素为止

数组:

9	3	1	-	3	*	+			
---	---	---	---	---	---	---	--	--	--

+

栈:

[illegible]

$$\begin{array}{c} \square \\ \square \\ \square \\ \square \\ \square \\ \square \\ \square \\ + \end{array}$$

[illegible]

9	3	1	-	3	*	+	10		
---	---	---	---	---	---	---	----	--	--

+

栈:



100

100

100

5

11



10

/

+

栈:



100

100

100

5



10

中缀变后缀

栈:

比如: $9 + (3 - 1) * 3 + 10 / 2$

(14) 字符串遍历结束, 栈内元素依次出栈

当字符串遍历结束时, 栈顶元素依次出栈, 直到栈为空为止。

后缀表达式为: **9 3 1 - 3 * + 10 2 / +**

数组:

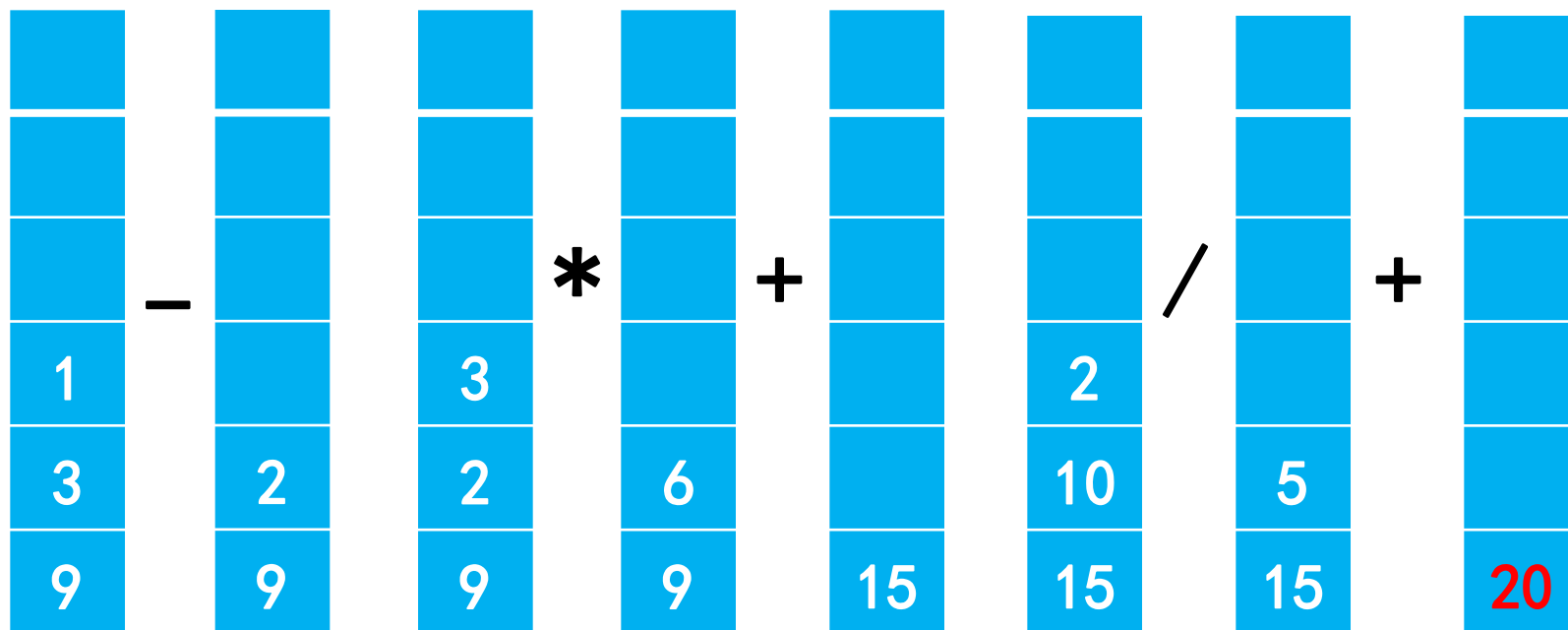
9	3	1	-	3	*	+	10	2	/	+
---	---	---	---	---	---	---	----	---	---	---



计算结果

后缀表达式为：9 3 1 - 3 * + 10 2 / +

计算规则：遇到数字，放入栈内，遇到运算符，计算栈顶元素和栈顶元素的下面一个元素，两个元素出栈，结果入栈，最终栈内的元素就是表达式的结果



后缀表达式

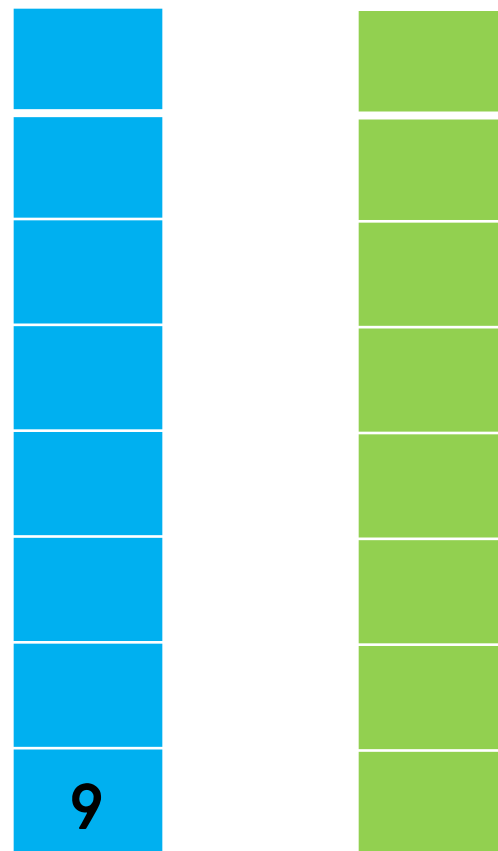
在后缀表达式中，先出现的运算符先计算，所以在把中缀表达式转换后缀表达式的时候，先计算的运算符肯定要比后计算的运算符先放入新的数组，所以这就是为什么当我们放入运算符的时候，如果当前运算符的优先级高于栈顶元素，入栈。而当前运算符的优先级不高于栈顶元素的时候，出栈的原因。而括号只是起到一个调节运算秩序的作用，所以在后缀表达式中就不需要了。

根据这种方法，我们也可以直接开两个栈计算，原理还是后缀表达式的原理。但是就不需要转换两次了。

表达式求值

比如： $9 + (3 - 1) * 3 + 10 / 2$

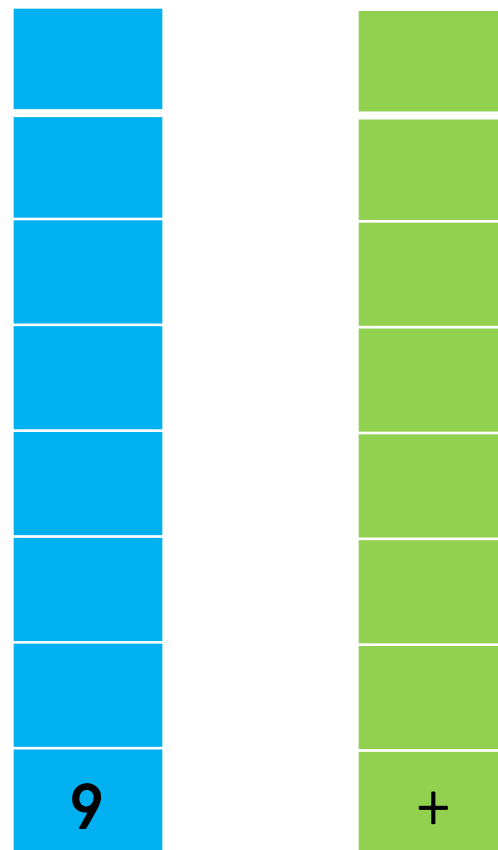
(1) 数字直接入数字栈



表达式求值

比如: $9 + (3 - 1) * 3 + 10 / 2$

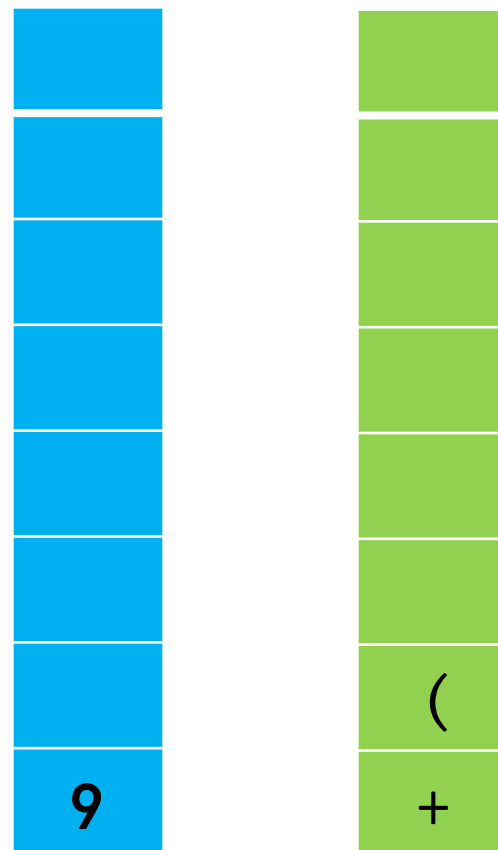
(2) “+”：此时符号栈为
空，“+”入栈



表达式求值

比如: $9 + (3 - 1) * 3 + 10 / 2$

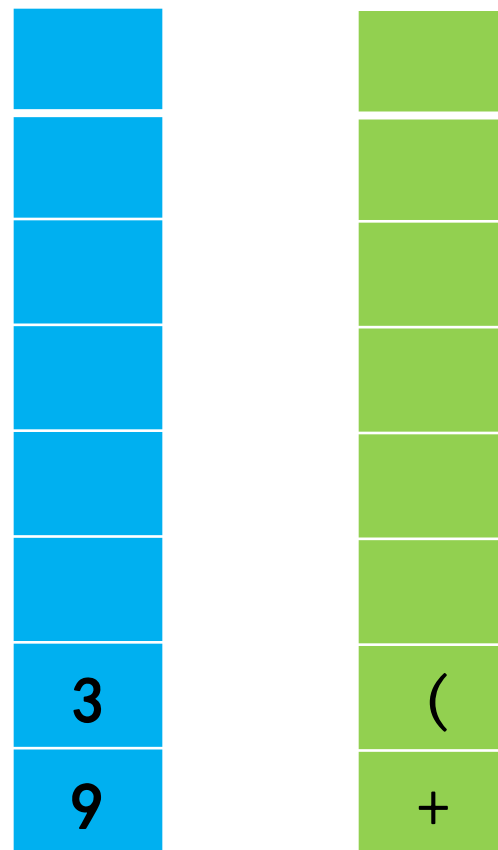
(3) “(” : 左括号直接入栈



表达式求值

比如: $9 + (3 - 1) * 3 + 10 / 2$

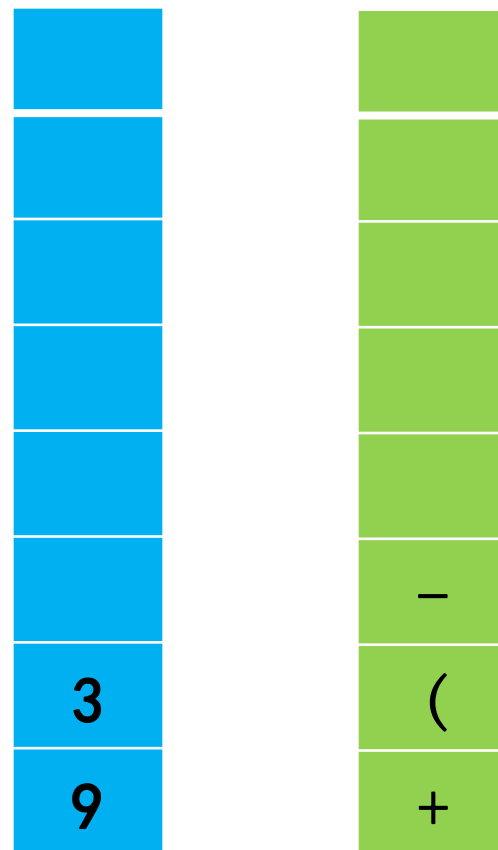
(4) 数字3: 数字直接入栈



表达式求值

比如： $9 + (3 - 1) * 3 + 10 / 2$

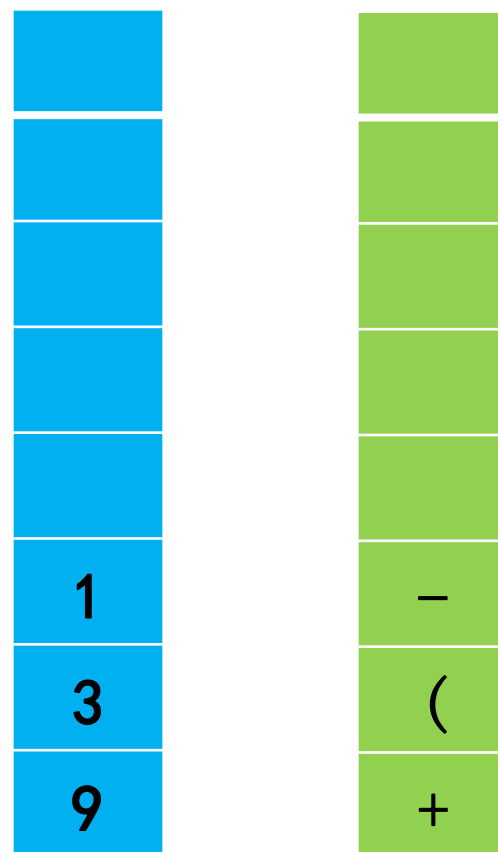
(5) “-”：此时栈顶元素是“（”，先算括号里面的，所以“-”入栈。栈顶的运算符先算。



表达式求值

比如: $9 + (3 - 1) * 3 + 10 / 2$

(6) 数字1: 数字直接入栈

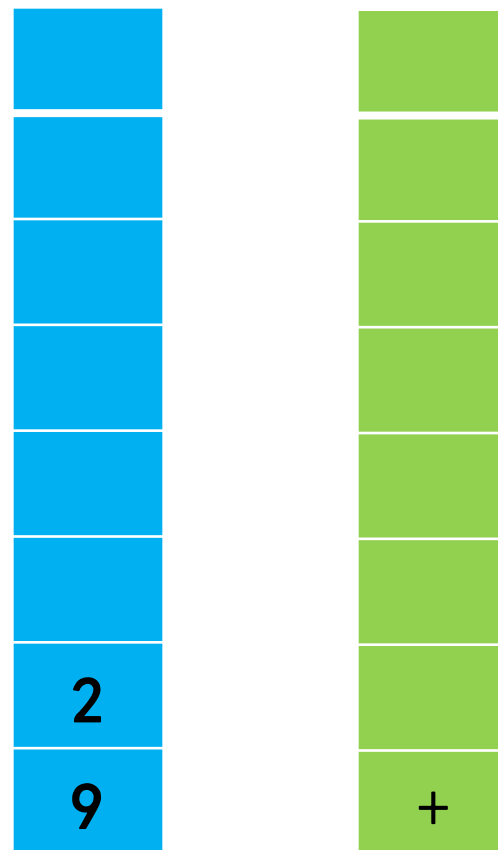


表达式求值

比如： $9 + (3 - 1) * 3 + 10 / 2$

(7) “)”：当遇到“)”的时候，出栈，直到栈顶元素为“(”为止。边出栈，边计算。因为“)”表示括号结束，可以计算括号里面的了

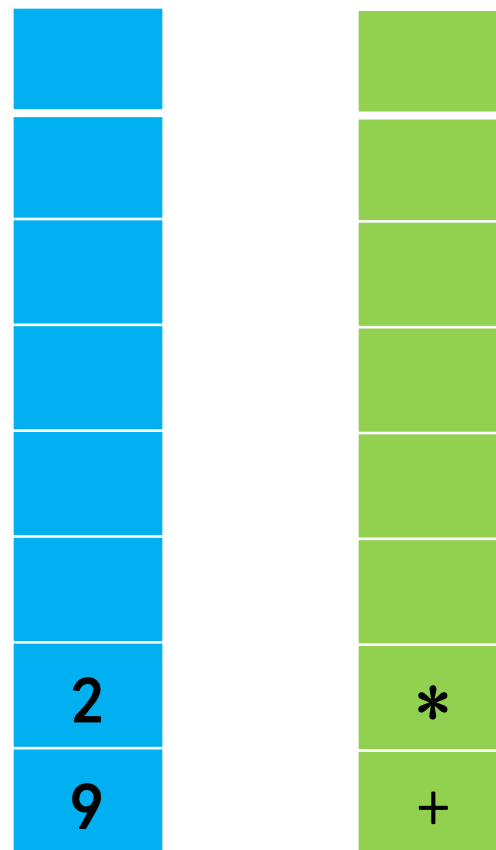
先“-”出栈，出栈的同时，数字栈的最上面两个数字相减，注意是下面一个减去上面一个。
 $3 - 1 = 2$, 2入栈



表达式求值

比如： $9 + (3 - 1) * 3 + 10 / 2$

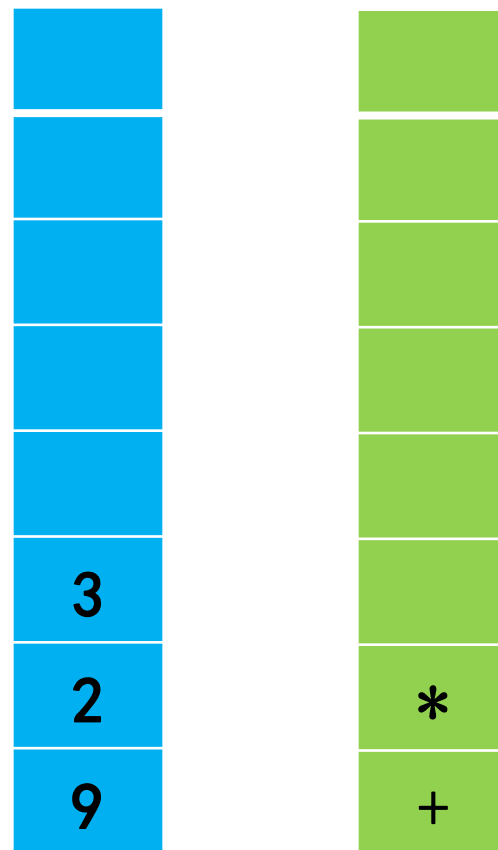
(8) “*”：栈顶元素是“+”，
“*”优先级高于“+”，“*”肯定先于“+”计算，“*”入栈。
(因为不知道后面有没有括号，所以“*”不能直接用于计算)



表达式求值

比如： $9 + (3 - 1) * 3 + 10 / 2$

(9) 数字3： 数字入栈



表达式求值

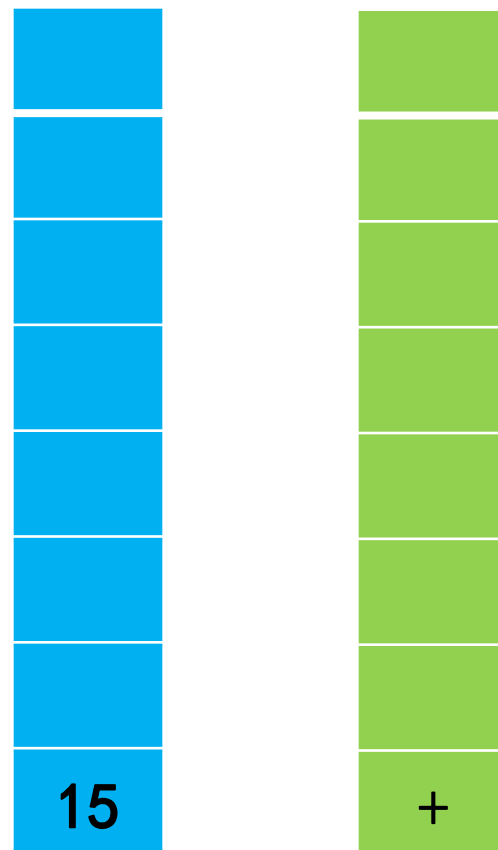
比如： $9+(3-1)*3+10/2$

(10) “+”：此时栈顶元素是“*”，出栈并计算。结果入栈

$3*2=6$ 6入栈

此时栈顶元素是“+”，先出现的“+”先计算，所以“+”先出栈，后一个“+”再入栈。

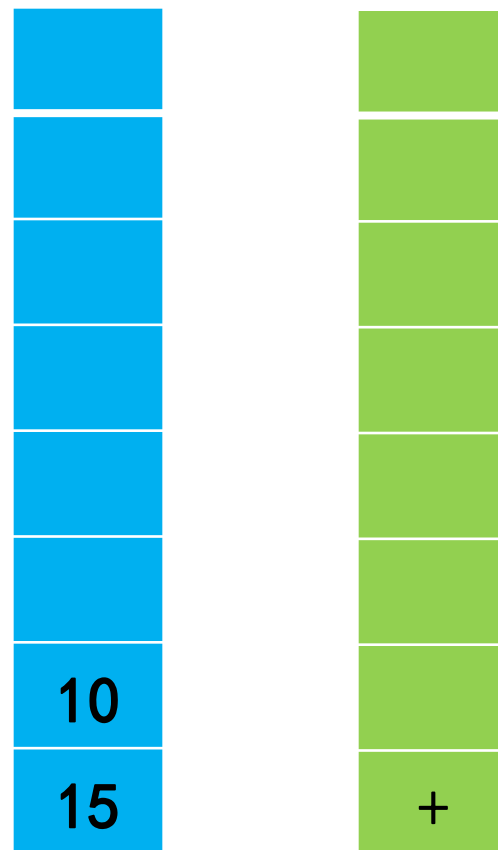
$9+6=15$ 15入栈



表达式求值

比如： $9 + (3 - 1) * 3 + 10 / 2$

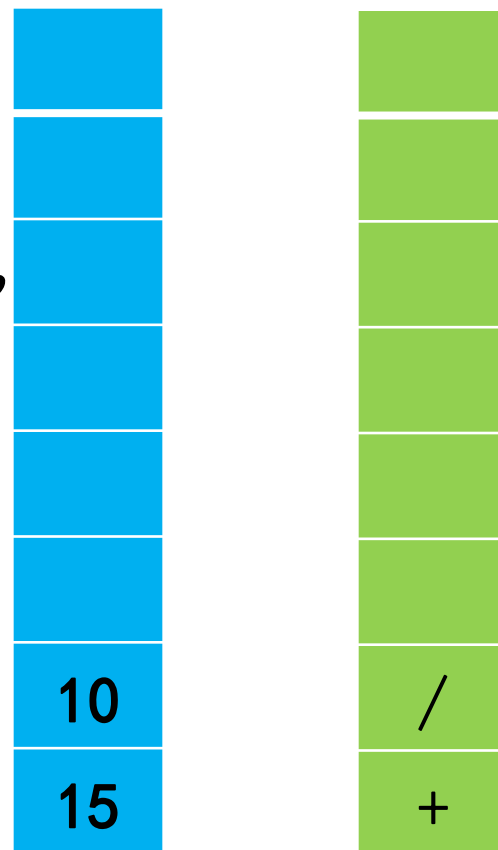
(11) 数字10，数字入栈



表达式求值

比如： $9 + (3 - 1) * 3 + 10 / 2$

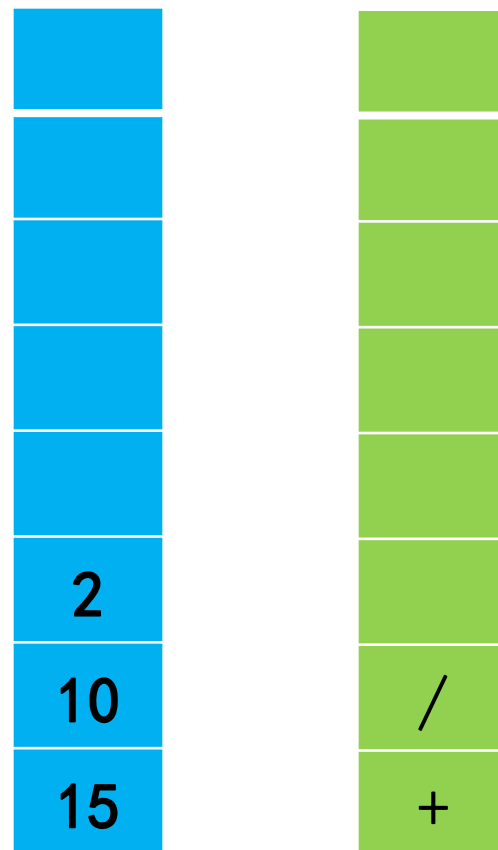
(12) “/”，“/”优先级高于“+”，说明先算“/”再算“+”，所以“/”入栈。



表达式求值

比如： $9 + (3 - 1) * 3 + 10 / 2$

(13) 数字2，数字入栈



表达式求值

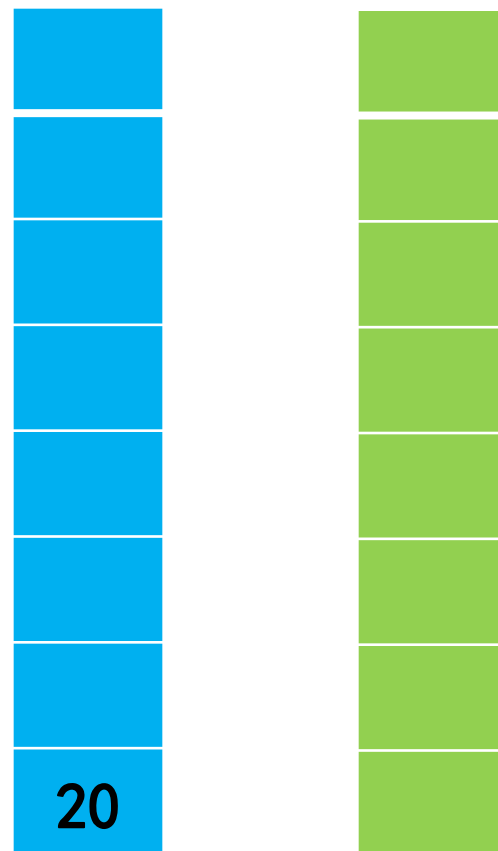
比如： $9 + (3 - 1) * 3 + 10 / 2$

(14) 此时字符串遍历结束，但是栈不为空，所以字符依次出栈，对应的数字也依次出栈。

“/” 出栈，2和10出栈，
 $10 / 2 = 5$ 5入栈

“+” 出栈，5和15出栈，
 $15 + 5 = 20$ 20入栈

最终结果：20



真题

表达式 $a*(b+c)*d$ 的后缀形式是()

A: a b c d * + *

B: a b c + * d *

C: a * b c + * d

D: b + c * a * d

前缀表达式

后缀表达式是运算符在数字后面，那么前缀表达式就很明显是运算符在数字前面，实际上规则和中缀转后缀差不多。

对于中缀表达式，先倒着读，按照类似中缀转后缀的方式转换完之和再倒序。

前缀表达式

比如 $1+2*3+(4*5+6)*7$

转换成前缀表达式，第一步先倒着读。

(1) 读入 '7'，并送到输出，然后 '*' 被读入并压入栈中。
接下来 ')' 读入并送到栈中，此时状态如下：

栈：*)

输出：7

(2) 接下来读入 '6'，并送到输出，然后读入 '+' 此时状态如下：

栈：*) +

输出：7 6

(3) 然后读入 '5'，并送到输出，然后继续读入 '*'，由于 '*' 的优先级比 '+' 高，所以直接加入栈中，再将 '4' 送到输出，因此状态如下：

栈：*) + *

输出：7 6 5 4

前缀表达式

(4) 下一个读入的符号 ‘(’，由于规则，因此将符号栈中的操作符依次出栈，然后读入 ‘4’，并在最后将右括号弹出：

栈：*

输出：7 6 5 4 * +

(5) 继续读入，此时读入 ‘+’，由优先级小于栈中 ‘*’ 的优先级，因此依照规则，依次出栈，最后将 ‘+’ 入栈，接下来读入 ‘3’：

栈：+

输出：7 6 5 4 * + * 3

(6) 往后读入的符号是 ‘*’，将 ‘*’ 入栈。然后将 ‘2’ 放入输出：

栈：+ *

输出：7 6 5 4 * + * 3 2

(7) 现在读入 ‘+’，由于符号栈中存在 ‘+’ 且它们的优先级相同，根据规则，除非，优先级大于栈顶元素，否则不出栈，依次栈中状态如下：

栈：+ +

输出：7 6 5 4 * + * 3 2 *

前缀表达式

(8) 下一个读入 '1' :

栈: + +

输出: 7 6 5 4 * + * 3 2 * 1

(9) 现在输入为空, 弹出所有栈中元素

栈: 空

输出: 7 6 5 4 * + * 3 2 * 1 + +

(10) 最后翻转

输出: + + 1 * 2 3 * + * 4 5 6 7

前缀表达式

前缀表达式转换成中缀表达式也差不多。

第一步，先倒序，剩下的和后缀转中缀差不多

比如：+ + 1 * 2 3 * + * 4 5 6 7

翻转：7 6 5 4 * + * 3 2 * 1 + +

遇到数字就放入栈内，遇到运算符就栈顶两个元素计算。

真题

表达式 $a*d-b*c$ 的前缀形式是 ()

A: $a\ d\ *\ b\ c\ *\ -$

B: $-*\ a\ d\ *\ b\ c$

C: $a\ *\ d\ -\ b\ *\ c$

D: $-*\ *\ a\ d\ b\ c$

真题

就算不会转换，只要知道什么是前缀，后缀，中缀，猜也能猜出来，既然是前缀，那么明显运算符在数字之前，所以肯定从B, D中选一个，D三个运算符连在一起，一看就不能先，所以肯定选B。

倒过来 $c*b-d*a$

(1) c

(2) cb

(3) cb*

(4) cb*d

(5) cb*da

(6) cb*da*

(7) cb*da*-

计算机网络

网络分为局域网，城域网，广域网。

TCP/IP协议是一种4层的协议方式，从上到下分为应用层，传输层，网络层，网络接口层。其中TCP属于传输层，IP属于网络层。

IP地址：

IPV4是32位二进制数，4个十进制数

每个十进制数的范围是0-255 (2^8-1)

每个数之间用小数点隔开

真题

由四个不同的点构成的简单无向连通图的个数是()

A:32 B:35 C:38 D:41

题解

该题如果手画，很麻烦，还是要用到排列组合的思想。
一共4个点，那么最多连 $4 \times (4-1) / 2 = 6$ 条边。要让图连通，
那么最少需要 $4-1$ 条边(树)。那么就分情况讨论：

3条边：共有 $C(6, 3)$ 种，但是三条边构成三角形这种是错误的，这种情况有4种

4条边：共有 $C(6, 4)$ 种，都满足条件

5条边：共 $C(6, 5)$ 种，都满足条件

6条边：共 $C(6, 6)$ 种，都满足条件

所以共 $20-4+15+6+1=38$ 种

真题

将7个名额分给4个班级，每个班至少有一个名额，有（ ）种不同的分配方案

A:35 B:15 C:20 D: 120

将7个名额分给4个班级，允许有班没有名额，有（ ）种不同的分配方案

A:60 B:84 C:96 D: 120

真题

第一题很简单，直接用隔板法就可以了，7个班，那么有6个空位，分成4份，那么3个隔板就够了，所以答案就是 $C(6, 3)=20$

第二题呢？还是这样做？明显不对，因为很有可能分成1, 2, 3, 4份，那么是 $C(6, 0)+C(6, 1)+C(6, 2)+C(6, 3)$ 吗？明显不是，因为班级是不一样的，还要算相同方案分配给不同班级的情况，很麻烦。

我们可以先增加4个，然后分配的时候保证每个班级至少一个，最后每个班再拿掉这一个就可以了。

$$C(10, 3)=120$$

捆绑法

对于某几个元素要求相邻的排列问题，可以先将相邻的元素捆绑在一起，看做一个大的元素，与其他元素排列，然后再对相邻的元素内部进行排列。

7个人站成一排照相，要求甲乙丙三人相邻，问有多少种排列方法：

我们先把甲乙丙三人捆绑到一起看做一个元素，与其余四人共五个元素做全排列，共 $A(5, 5)$ 种排法，再对甲乙丙三人进行全排列 $A(3, 3)$ ，乘起来就是答案。

9个人排成一排，有两个个老师，3个男学生，4个女学生，三个男学生要求相邻，4个女学生也要求相邻，问有多少种排列方法。

除法

对于某几个元素顺序一定的排列问题，可以先将这几个元素与其他元素一同进行排列，然后用总的排列数除以这几个元素的全排列数

7人排队，甲在乙前面的排法有几种：

先不考虑限制条件，有 $A(7, 7)$ 种可能，而甲乙之间的排法有 $A(2, 2)$ 种，甲在乙前面的排法只有一种复合条件，所以直接用 $A(7, 7) / A(2, 2)$ 就可以了。

直排法

如果若干个元素需要排成几排，问有多少种排法。比如7个人，第一排3人，第二排4人，问有多少种不同的排法。
实际上和排成一排，有七个人是一样的，答案是 $A(7, 7)$ 。

消序

有4名男生，3名女生，高矮互不相等，将他们排成一排，要求从左到右，女生由矮到高排，问有多少种排法。

其实很简单，女生从矮到高只有一种排法，男生随便排就可以了，最终答案 $A(7, 4)$

隔板法

把 n 个相同的小球放入 m 个不同的盒子，不允许盒子为空，但是球必须放完，问有多少种不同的方法。

我们可以用隔板法，相当于用 $m-1$ 个隔板把 n 堆小球分成 n 堆，其中有 $n-1$ 个位置可以选择，所以答案为 $C(n-1, m-1)$

但是隔板法不能用于有盒子为空这种情况，如果可以盒子可以为空，那么很明显不成立。那该怎么办呢？

我们转换一下，加上 m 个小球，保证每个盒子都有一个小球，这样就满足隔板法的要求了(最后每个盒子去掉一个小球就是满足原始条件的答案了)，所以有 $C(n+m-1, m)$ 种情况

处理排列组合问题的方法还有很多，自行研究。

排序

排序有很多种，效率也各不相同。

N^2 : 插入排序，冒泡排序，选择排序

$N \log N$: 归并排序，堆排，快速排序，希尔排序

对于 $N \log N$ 的排序，有的效率是比较稳定的，即无论什么情况都趋近于 $N \log N$ ，比如归并排序，堆排，也有一些排序时间复杂度是不稳定的，即在最坏情况下会趋近于 n^2 ，比如快速排序，希尔排序

排序

排序也分为稳定排序和不稳定排序，这个稳定与否不是指排序效率的，而是指排序前后两个相等的数相对位置一定不发生改变，则算法稳定。

冒泡排序：稳定排序，比较相邻两个元素大小，不满足大小关系就交换。

选择排序：不稳定排序。比如5 8 5 2 9，第一轮比较之后2和5交换位置，相对位置就变化了。

插入排序：稳定排序，参考一张一张砌牌。

排序

快速排序：不稳定排序，很容易想到，比如5 2 2 7 7，第一个2和第二个7交换，第二个2和第一个7交换，位置变了。

归并排序：稳定排序，很明显，自己脑补

希尔排序：虽然是插入排序的升级版，但是 he 是不稳定的排序。要学的自己查资料

基数排序：后缀排序的时候讲过，很明显是稳定的

堆排：不稳定排序，这种直接交换的多半是不稳定的

语言

程序语言也可以分为机器语言，汇编语言，高级语言。机器语言，计算机能识别的语言，只要就是由0,1组成，高级语言结构类似人们所熟悉的语言，比如英文，中文，等等，所以我们用的大都是高级语言。汇编语言介于两者之间

时间复杂度

6. 若某算法的计算时间表示为递推关系式：

$$T(N) = 2T(N/2) + N \log N$$

$$T(1) = 1$$

则该算法的时间复杂度为()。

A. $O(N)$ B. $O(N \log N)$

C. $O(N \log^2 N)$ D. $O(N^2)$

时间复杂度

求时间复杂度的题在初赛中也常考内容，不过都有点不好算，但是一般还是可以猜一个大概，比如这道题，很容易才出有一个 $\log$ ，但是B,C中到底选哪一个呢？直接不好求，那就先算出 $T(1)$, $T(2)$, $T(3)$ 。。。然后再看和哪个时间复杂度趋近。

数列求解方法

1: 类似于 $f(n)=f(n-1)+n$ 这种递推公式

可以用错位相减来求解

$$f(n-1)=f(n-2)+(n-1)$$

...

$$f(2)=f(1)+2$$

每一项左边相加，右边相加，然后同项一减，就得到

$f(n)=f(1)+\text{常数项}$ 一般 $f(1)$ 或者 $f(0)$ 给出了

这种时间复杂度为 n^2 。

数列求解方法

2: 类似于 $f(n)=a*f(n-1)+b$ 这种递推公式

重新构造一个数列, 形式为

$$f(n)-x=a(f(n-1)-x)$$

求解出 x 的值

这样 $f(n)-x$ 就是一个等比数列了, 就可以求出 $f(n)-x$ 的值了, 同样, 也可以求出 $f(n)$ 的值

数列求解方法

3: 类似于 $f(n)=f(n-1)+f(n-2)$ 这种递推公式(每一项前有系数)

方法叫什么名字不记得了, 大概就是重新构造数列

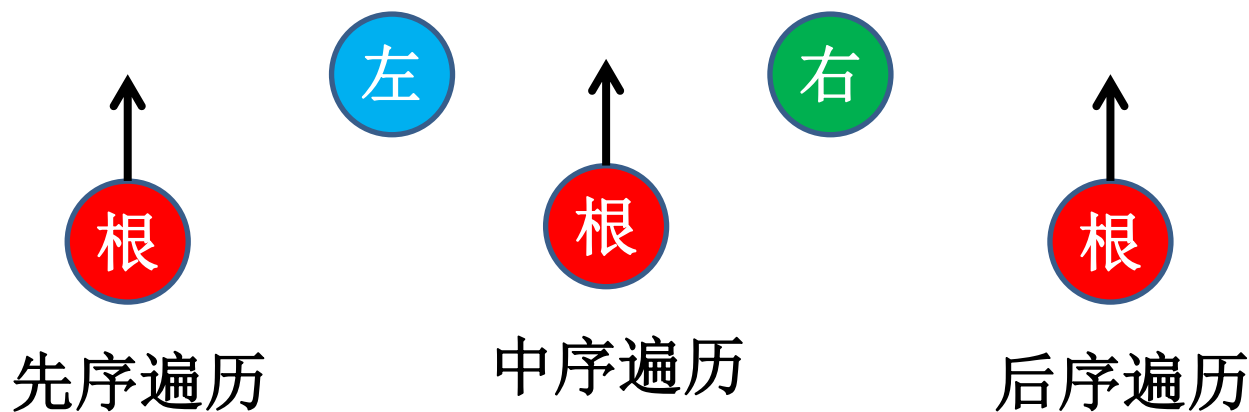
$$f(n)+af(n-1)=b(f(n-1)+bf(n-2))$$

$f(n)=f(n-1)+f(n-2)$ 。求出 a 和 b

就可以转换成 $f(n)+af(n-1)=m$, 再按照前面的方法转换就可以了。

二叉树的遍历

二叉树的遍历分为先(前)序，中序，后序遍历，这里的先，中，后都是指根的遍历顺序，左右子树的遍历顺序永远都是先左后右。



二叉树的遍历

已知二叉树的先序遍历，中序遍历，可以得出树的形态，当然也就可以得到树的后序遍历。

已知二叉树的中序遍历，后序遍历，可以得出树的形态，当然也就可以得到树的先序遍历

但是已知树的先序遍历，后序遍历，**无法**确定树的形态，也就没有办法得到中序遍历。

已知树的两种遍历结果，如何得到树的形态呢？其实很简单，递归得去找根就可以了。因为只有中序遍历可以确定根的位置，所以两种遍历中必须有中序遍历。

二叉树的遍历

已知二叉树的先序遍历，中序遍历，可以得出树的形态，当然也就可以得到树的后序遍历。

已知二叉树的中序遍历，后序遍历，可以得出树的形态，当然也就可以得到树的先序遍历

但是已知树的先序遍历，后序遍历，**无法**确定树的形态，也就没有办法得到中序遍历。

已知树的两种遍历结果，如何得到树的形态呢？其实很简单，递归得去找根就可以了。因为只有先序和后序遍历可以确定根的位置，只需要其中一个就可以了，所以只知道先序和后序是无法确定树的形态的

二叉树的遍历

已知一棵二叉树的先序遍历序列为ABGDECFH，中序遍历序列为GBEDAFCH，则其后序遍历序列为？

二叉树的遍历

先序：ABGDECFH

中序：GBEDAFCH

一：在先序中确定整颗树的根(A), 先序遍历第一个遍历到的肯定是整颗树的根结点，在中序中确定哪些结点在根的左子树，哪些结点在根的右子树，很明显，(GBED)在根(A)左边，所以是左子树，(FCH)在根(A)右边，所以是右子树。

二叉树的遍历

先序：ABGDECFH

中序：GBEDAFCH

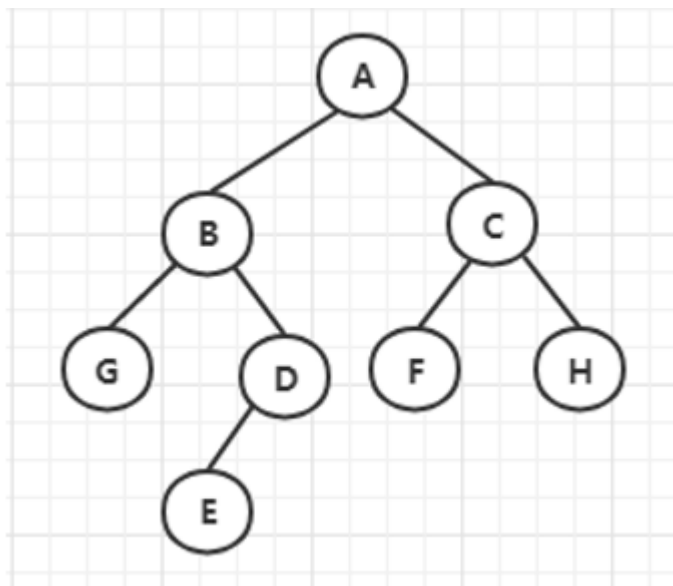
二：接下来怎么办呢？很简单，既然你都知道了(GBED)是一棵完整的左子树，那么你再去确定树(GBED)的根就好了，根去哪确定呢？继续去先序遍历中确定，(GBED)中第一个出现的字母(B)就是该子树的根，同样对于右子树(FCH)，在先序遍历中第一个出现是C，所以C是(FCH)的根。中序遍历中F在C左边，所以是左子树，H在C右边，所以是右子树。剩下的自己去划分。

二叉树的遍历

先序：ABGDECFH

中序：GBEDAFCH

最终可以画出来树的形态是这样的：
所以后序遍历为：GEDBFHCA



二叉树的遍历

已知一棵二叉树的中序遍历序列为GBEDAFCH，后序遍历序列为GEDBFHCA，则其先序遍历序列为？

二叉树的遍历

中序: GBEDAFCH

后序: DEDBFHCA

一样的做法，后序遍历确定根，在后序遍历中整棵树的根是最后一个出现的，所以A为根，再去中序遍历中去以A为界限划分左右子树，左子树(GBED)，右子树(FCH)，一直递归做下去直到该区间只有一个结点为止。

二叉树的遍历

中序: GBEDAFCH

后序: DEDBFHCA

一样的做法，后序遍历确定根，在后序遍历中整棵树的根是最后一个出现的，所以A为根，再去中序遍历中去以A为界限划分左右子树，左子树(GBED)，右子树(FCH)，一直递归做下去直到该区间只有一个结点为止。

二叉树的遍历

中序: GBEDAFCH

后序: DEDBFHCA

一样的做法，后序遍历确定根，在后序遍历中整棵树的根是最后一个出现的，所以A为根，再去中序遍历中去以A为界限划分左右子树，左子树(GBED)，右子树(FCH)，一直递归做下去直到该区间只有一个结点为止。

树的性质

对于任意一棵二叉树，叶子结点个数为 n_0 ，度为2(儿子个数为2)的结点个数为 n_2 ，那么一定有 $n_0=n_2+1$ 。

证明：

设度为1的结点个数为 n_1 ，总结点个数为 n

$$n=n_0+n_1+n_2$$

如果我们根据儿子个数来看，度为1的结点有1个儿子，度为2的结点有2个儿子，根结点不是任何结点的儿子

$$n=n_1+2*n_2+1$$

两个等式消去 n 和 n_1 ，得到了

$$N_0=n_2+1。$$

树的性质

对于任意一棵二叉树，叶子结点个数为 n_0 ，度为2(儿子个数为2)的结点个数为 n_2 ，那么一定有 $n_0=n_2+1$ 。

证明：

设度为1的结点个数为 n_1 ，总结点个数为 n

$$n=n_0+n_1+n_2$$

如果我们根据儿子个数来看，度为1的结点有1个儿子，度为2的结点有2个儿子，根结点不是任何结点的儿子

$$n=n_1+2*n_2+1$$

两个等式消去 n 和 n_1 ，得到了

$$N_0=n_2+1。$$

栈

有 n 个数依次进栈，可以随机出栈，求出栈的所有可能情况。

输入：

3

输出：

5

当然，这是一道卡特兰数的题，但是：

(1) 万一你不记得卡特兰数的公式了呢

(2) 万一你不知道这是卡特兰数呢

所以，背公式还不如自己推导呢！

卡特兰数

简单来说，就是我们已经知道了 $1 \sim i-1$ 个数出栈的可能性想，需要我们求出 i 个数出栈的可能性。

假设此时第 k 个数已经入栈了，那么还剩下 $(n-k)$ 个数，那么剩下 $(n-k)$ 个数的入出栈方案数很简单就是 $h[n-k]$ ，那么已经入栈的 k 个数的方案数呢？ $h[k]$ ？肯定不是， $h[k]$ 表示的是前 k 个数出入栈种树，但是这里已经入栈的 k 个元素，有没有必要先出栈，再让 $k+1$ 这个元素进来，所以，明显有问题。

那我们就换一种思路，假设最后一个出栈的数是 k ，那么说明 $[1 \cdots k-1]$ 都在 k 入栈之前出栈了，然后 k 再入栈，接下来再是 $[k+1 \cdots n]$ 入栈，然后 $[k+1 \cdots n]$ 出栈。那么此时 k 都把前后两部分隔离成了独立的两部分，在 k 入栈之前，前面部分已经入栈并且出栈了。在 k 之后的部分，后面部分比 k 后入栈并且先入栈。所以，公式就出来了：

$$h[n] = h[0] * h[n-1] + h[1] * h[n-2] + \cdots + h[n-1] * h[0]$$

卡特兰数

还有一种证明方法，也很简单：

设 $h[i][j]$ 表示栈内有 i 个，出栈的有 j 个的方案数。要达到当前这种状态，他的前一状态只有两种可能，入栈一个或者出栈一个，如果是入栈要到达 $h[i][j]$ ，那么上一状态就是 $h[i-1][j]$ ，如果是出栈到达 $h[i][j]$ ，那么就是 $h[i+1][j-1]$ 。那么递推公式就有了：

$$h[i][j] = h[i-1][j] + h[i+1][j-1].$$

注意一下，只有 j 是递增的，可能后用到后面的值，所以先枚举 j ，在枚举 i 。就可以得到正确答案。

但是前面两种时间复杂度都是 $O(n^2)$ 的，能不能更快一点，直接找公式而不是递推公式。

卡特兰数

公式也是可以推导的。画一个图就很好理解了。

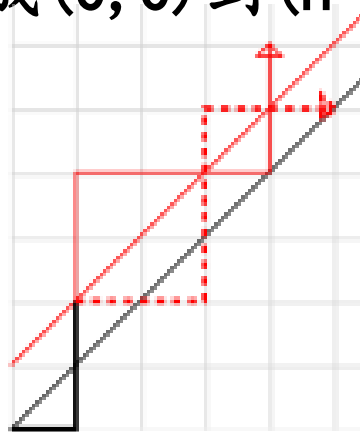
在平面直角坐标系中，x轴表示入栈次数，y轴表示出栈次数，那么卡特兰数就可以表示为从(0,0)点到达(n,n)点不越过直线 $y=x$ 的方案数。

如果不考虑限制，总方案数为 $C(2n, n)$ ，那么限制呢？我们会发现对于所有不满足条件的情况，一定是在若干次操作之出栈比入栈多一次，即这个点在 $y=x+1$ 上，我们发现如果把后面部分都相对于这条线对称，那么最后一定要到达点 $(n-1, n+1)$ 。即任何非法路径都可以变成(0,0)到 $(n-1, n+1)$ 的一条无限制的情况，那么合法方案为：

$C(2n, n) - C(2n, n-1)$ ：

也可以转换成 $1/(n+1) * C(2n, n)$

还有 $h[i+1] = (4n+2)/(n+2) * h[i]$



卡特兰数

根据前面的推导过程，我们很容易想到拉特兰数的适用范围：

(1) 一种情况的数量一直 $\geq$ 另一种情况

(2) 最终两种情况的数量一定相等

只要满足这两种条件，那么他就是卡特兰数

(1) 给出一个 n ，要求一个长度为 $2n$ 的01序列，使得序列的任意前缀中的1的个数不少于0的个数

(2) 从 $(0, 0)$ 到达 (n, n) 点，每次只能向上或者向右，问不超过 $y=x$ 的方案数

(3) 有 $2n$ 个人进入剧场，入场费5元，售货员自己没零钱有 n 个人有5元的钞票，剩下 n 个人只有10元钞票，问有多少种方法可以使得所有人都可以正常入场。

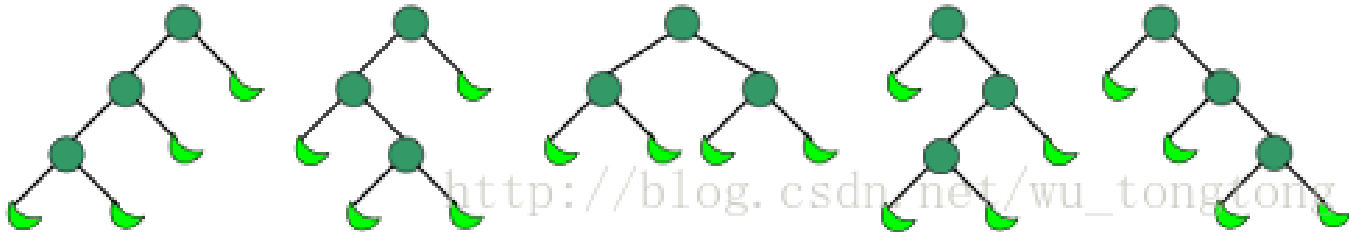
(4) n 个左括号， n 个右括号，问合法括号排列的方案数

卡特兰数

一共有 $2n+1$ 个结点，问所有不同构的含有 $(n+1)$ 个叶子结点的二叉树的个数(区分左右儿子)。

对于一个满二叉树而言，叶子结点个数=非叶子结点数+1
但是实际上，也有不是满二叉树就满足条件。

只要满足子树要么为空，要么满都可以满足条件。设叶子结点为 x ，非叶子结点为 y ；如果子树都满或者空， $x=y+1$ ，如果不满足， $x < y+1$ ；并且只有当插入到只有一个儿子结点的位置，叶子结点才会+1。所以一定是满或空这种形态。



卡特兰数

那么我们思考一下，最开始只有一个结点，就是叶子结点，然后再添加一个结点，此时 $x=y$ ，那么为了满足 $x=y+1$ ，又的让叶子结点+1。一直这样循环。当最后满足有 n 个叶子结点， n 个非叶子结点的时候，最后再添加的那一个结点是固定的，所以实际上还是卡特兰数。只是：

$h[i]$ 表示有 $n+1$ 个叶子结点的二叉树的个数。